

Natural Language Processing 2

Sequence to Sequence Modeling

Fatemeh Dehrouyeh



FANSHAWE

Contents

1 Introduction

2 Encoder-Decoder Framework

Current Section

1 Introduction

2 Encoder-Decoder Framework

Why Sequence-to-Sequence Modeling Matters

- Many real-world problems involve turning one sequence into another.
- A classic example: **Machine Translation** - Converting sentences from one language to another (e.g., English $\rightarrow$ French).
- But the idea goes far beyond languages:
 - Converting Python code into Java.
 - Summarizing a long article into a few sentences.
 - Generating a melody from a sequence of notes.
- All of these share a common principle: **map input sequence $\rightarrow$ output sequence**.

Seq2Seq: Human vs. Machine Translation

Human Translator:

$$y^* = \arg \max_y p(y|x)$$

- Uses intuition: “Which translation sounds most natural?”
- Example: $x = \text{“I am hungry”}$
 $y = \text{“J’ai faim”}$

Machine Translator:

$$y' = \arg \max_y p(y|x, \theta)$$

- Learns a scoring function $p(y|x, \theta)$.
- θ = model parameters learned from data.
- Chooses the translation with the highest probability.

Current Section

1 Introduction

2 Encoder-Decoder Framework

Encoder-Decoder Framework

- A common practice for sequence-to-sequence models is the **encoder-decoder** setup.
- **Encoder:** processes the input sequence and compresses its meaning into a hidden representation.
- **Decoder:** takes that hidden representation and unfolds it into the target sequence.

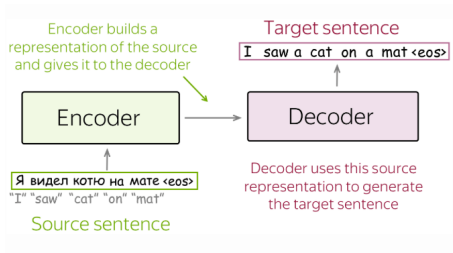


figure source: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Intuition Behind the Model

- Think of the encoder as **packing the whole input into a suitcase**.
- The decoder then **unpacks the suitcase**, word by word, into the target language.
- Limitation: one fixed-size vector must capture **all** details of the input — this works for short sentences but struggles with long or complex ones.

Simplest LMs: Two RNNs for Encoder and Decoder

- The most basic encoder–decoder design uses **two RNNs** (often LSTMs).
- **Encoder RNN:** processes the source sentence word by word, updating its hidden state.
- After the last word, the encoder's final state becomes a **compressed summary vector**.
- **Decoder RNN:** starts from this summary vector and generates the target sequence step by step.

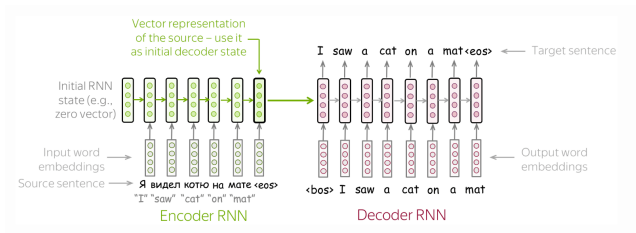
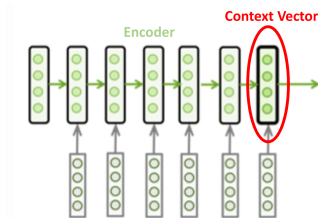


figure source: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Encoder Context Vector

- The encoder reads the full input sequence.
- At the final time step, it outputs its final hidden state.
- This final hidden state is used as the **context vector**.
- The decoder uses this context vector to start generating the output sequence.

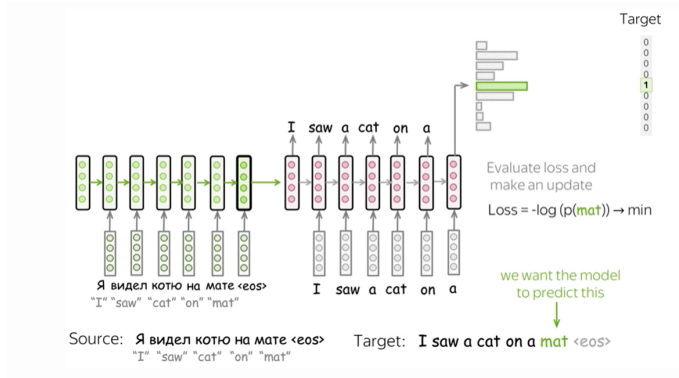


For LSTMs

The context usually includes both the final hidden state and the final cell state.

How LMs Work in Seq2Seq: Training

- Goal: at each decoding step, predict the correct next word.
- The model outputs a probability distribution over the vocabulary.
- We want the correct word to have the highest probability.
- Training uses the same idea as in classification tasks: **cross-entropy loss**.



Cross-Entropy Loss

Cross-entropy loss measures how far the model's predicted probabilities are from the correct target word.

At each decoding step, the model outputs a probability distribution over the vocabulary:

$$\hat{y} = [P(w_1), P(w_2), \dots, P(w_V)]$$

where V is the vocabulary size.

For one target word, cross-entropy is:

$$\mathcal{L} = - \sum_{i=1}^V y_i \log(\hat{y}_i)$$

Since the correct word has $y_i = 1$ and all other words have $y_i = 0$, this becomes:

$$\mathcal{L} = - \log(P(\text{correct word}))$$

Key Idea

The loss becomes small when the model assigns high probability to the correct next word.

How LMs Work in Seq2Seq: Prediction (Big Picture)

During prediction, the encoder first reads the full source sentence.

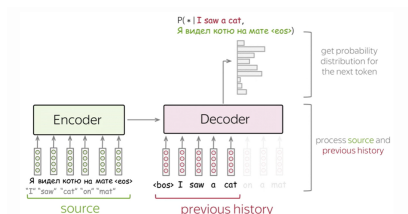
Then the decoder generates the target sentence one token at a time.

source sentence $\rightarrow$ encoder $\rightarrow$ decoder $\rightarrow$ target tokens

At each decoding step, the decoder uses:

- the encoded source sentence,
- the previously generated target tokens,

to predict the next token.



Key Idea

Prediction is step-by-step: the decoder predicts one next token, adds it to the history, and repeats.

Predicting the Next Token

At each step, the decoder produces a vector representation of the current context.

This context includes:

- the encoded source sentence,
- the previous target tokens.

Then a Dense layer maps this context vector to vocabulary-sized scores:

context vector $\rightarrow$ scores for all vocabulary tokens

Softmax converts these scores into probabilities:

$$P(w_1), P(w_2), \dots, P(w_V)$$

where V is the vocabulary size.

Key Idea

The decoder does not directly output a word. It outputs probabilities over all possible next tokens.

During prediction:

- The decoder starts with a start token.
- It predicts the next token.
- The predicted token is fed back into the decoder.
- This repeats until an end token is generated.

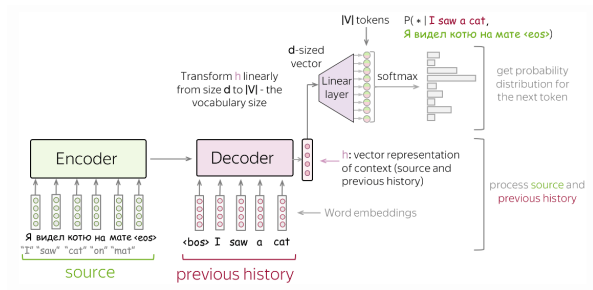


figure source: Seq2Seq and Attention

Let's See Some Codes!

Dataset for Machine Translation

A popular translation task:

English sentence $\rightarrow$ French sentence

Classic model:

Encoder LSTM $\rightarrow$ Context Vector $\rightarrow$ Decoder LSTM

Dataset:

English-French Translation Dataset

The dataset is huge, so let's create some samples manually.

Seq2Seq Coding: Translation Example

```
import tensorflow as tf
import numpy as np
import re
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, LSTM, Dense

# 1. Small English-French dataset for teaching
sentence_pairs = [
    ("i am happy", "je suis heureux"),
    ("i am sad", "je suis triste"),
    ("i am tired", "je suis fatigued"),
    ("i am hungry", "j'ai faim"),
    ("i am thirsty", "j'ai soif"),
    ("i love you", "je t'aime"),
    ("i hate you", "je te deteste"),
    ("thank you", "merci"),
    ("good morning", "bonjour"),
    ("good night", "bonne nuit"),
    ("how are you", "comment allez vous"),
    ("where are you", "ou etes vous"),
    ("what is your name", "comment tu t'appelles"),
    ("my name is john", "je m'appelle john"),
    ("i live here", "je vis ici"),
    ("come here", "viens ici"),
    ("go away", "va t'en"),
    ("sit down", "assieds toi"),
    ("stand up", "leve toi"),
    ("open the door", "ouvre la porte"),
    ("close the door", "ferme la porte"),
    ("i need help", "j'ai besoin d'aide"),
    ("help me", "aide moi"),
    ("i understand", "je comprends"),
    ("i do not understand", "je ne comprends pas"),
    ("i know", "je sais"),
    ("i do not know", "je ne sais pas"),
    ("this is good", "c'est bon"),
    ("this is bad", "c'est mauvais"),
    ("the weather is good", "il fait beau"),
    ("the weather is bad", "il fait mauvais"),
    ("i like this movie", "j'aime ce film"),
    ("this movie is good", "ce film est bon"),
    ("this movie is bad", "ce film est mauvais"),
    ("i want water", "je veux de l'eau"),
    ("i want food", "je veux de la nourriture"),
    ("i want coffee", "je veux du cafe"),
    ("i want tea", "je veux du the"),
    ("he is happy", "il est heureux"),
    ("she is happy", "elle est heureuse"),
    ("he is sad", "il est triste"),
    ("she is sad", "elle est triste"),
    ("we are ready", "nous sommes prêts"),
    ("they are ready", "ils sont prêts"),
    ("i am ready", "je suis prêt"),
    ("are you ready", "etes vous prêt"),
    ("see you later", "à plus tard"),
    ("see you tomorrow", "à demain"),
    ("i am learning", "j'apprends"),
    ("i am learning french", "j'apprends le français")
]
```

]

Seq2Seq Coding: Translation Example—Continued

```
# 2. Split English and French sentences
english_sentences = [pair[0] for pair in sentence_pairs]
french_sentences = [pair[1] for pair in sentence_pairs]

# 3. Clean text
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

english_sentences = [clean_text(sentence) for sentence in english_sentences]
french_sentences = [clean_text(sentence) for sentence in french_sentences]

# 4. Add start and end tokens to French target sentences
french_sentences = ["startseq " + sentence + " endseq" for sentence in french_sentences]

# 5. Tokenize English sentences
english_tokenizer = Tokenizer(oov_token="<OOV>")
english_tokenizer.fit_on_texts(english_sentences)

encoder_input_seq = english_tokenizer.texts_to_sequences(english_sentences)

english_vocab_size = len(english_tokenizer.word_index) + 1
max_encoder_len = max(len(seq) for seq in encoder_input_seq)

encoder_input_data = pad_sequences(encoder_input_seq, maxlen=max_encoder_len, padding="post")

# 6. Tokenize French sentences
french_tokenizer = Tokenizer(oov_token="<OOV>")
french_tokenizer.fit_on_texts(french_sentences)

decoder_full_seq = french_tokenizer.texts_to_sequences(french_sentences)

french_vocab_size = len(french_tokenizer.word_index) + 1
max_decoder_len = max(len(seq) for seq in decoder_full_seq)

decoder_full_data = pad_sequences(decoder_full_seq, maxlen=max_decoder_len, padding="post")
```

Why Add Start and End Tokens?

```
# 4. Add start and end tokens to French target sentences
french_sentences = ["startseq " + sentence + " endseq" for sentence in french_sentences]
```

We add special tokens to the French target sentence:

startseq je suis heureux endseq

startseq tells the decoder where to begin generating.

endseq tells the decoder when to stop generating.

During training:

Decoder input: startseq je suis heureux

Decoder target: je suis heureux endseq

Key Idea

The decoder learns to start from startseq and stop when it predicts endseq.

They are shifted by exactly one position. What the decoder is learning: At every time step, the decoder receives the current word and tries to predict the next word.

encoder_input_seq is the English sentences converted into integer IDs

▶ encoder_input_seq

```
... [[2, 4, 12],  
     [2, 4, 13],  
     [2, 4, 29],  
     [2, 4, 30],  
     [2, 4, 31],  
     [2, 32, 5],  
     [2, 33, 5],  
     [34, 5],  
     [6, 35],  
     [6, 36],  
     [37, 7, 5],  
     [38, 7, 5],  
     [39, 3, 40, 16],  
     [41, 16, 3, 42],  
     [2, 43, 17],  
     [44, 17],  
     [45, 46],  
     [47, 48],  
     [49, 50],  
     [51, 9, 18],  
     [52, 9, 18],
```

Why Do We Need Padding?

Sentences usually have different lengths.

[1, 2, 7] length 3

[1, 8, 9, 10] length 4

Neural networks train in batches, and each batch needs a fixed rectangular shape.

So we add zeros to shorter sentences:

[1, 2, 7, 0]

[1, 8, 9, 10]

Why +1 in vocabulary size?

```
english_vocab_size = len(english_tokenizer.word_index) + 1
```

We add +1 because token ID 0 is reserved for padding.

Key Idea

Padding makes all sequences the same length, and `mask_zero=True` tells the LSTM to ignore token ID 0.

Seq2Seq Coding: Translation Example—Continued

```
# 7. Create decoder input and decoder target
# Example:
# Full French:      startseq je suis heureux endseq
# Decoder input:    startseq je suis heureux
# Decoder target:   je suis heureux endseq
decoder_input_data = decoder_full_data[:, :-1]
decoder_target_data = decoder_full_data[:, 1:]

print("Encoder input shape:", encoder_input_data.shape)
print("Decoder input shape:", decoder_input_data.shape)
print("Decoder target shape:", decoder_target_data.shape)
print("English vocab size:", english_vocab_size)
print("French vocab size:", french_vocab_size)
print("Max encoder length:", max_encoder_len)
print("Max decoder length:", max_decoder_len)

# 8. Model parameters
embedding_dim = 64
lstm_units = 128

# 9. Encoder LSTM
encoder_inputs = Input(shape=(max_encoder_len,))

encoder_embedding = Embedding(input_dim=english_vocab_size, output_dim=embedding_dim, mask_zero=True)(encoder_inputs)

encoder_lstm = LSTM(lstm_units, return_state=True)

encoder_outputs, state_h, state_c = encoder_lstm(encoder_embedding)

encoder_states = [state_h, state_c]
```

Data Shapes for the Translation Model

```
Encoder input shape: (50, 4)
Decoder input shape: (50, 6)
Decoder target shape: (50, 6)
English vocab size: 65
French vocab size: 83
Max encoder length: 4
Max decoder length: 7
```

This means:

- We have **50 sentence pairs**.
- Each English input sentence is padded to **4 tokens**.
- Each decoder input sentence is padded to **6 tokens**.
- Each decoder target sentence also has **6 tokens**.

Key Idea

The encoder reads English token IDs, while the decoder learns to generate French token IDs one step ahead.

Why Translation Uses the Functional API

<pre># 12. Build LSTM model model = Sequential([Input(shape=(max_len,)), Embedding(input_dim=vocab_size, output_dim=embedding_dim, weights=[embedding_matrix], mask_zero=True, trainable=False), LSTM(64, activation="tanh"), Dropout(0.5), Dense(1, activation="sigmoid")])</pre>	Sequential Model
--	------------------

<pre># 0. Encoder LSTM encoder_inputs = Input(shape=(max_encoder_len,)) encoder_embedding = Embedding(input_dim=english_vocab_size, output_dim=embedding_dim, mask_zero=True)(encoder_inputs) encoder_lstm = LSTM(lstm_units, return_state=True) encoder_outputs, state_h, state_c = encoder_lstm(encoder_embedding) encoder_states = [state_h, state_c]</pre>	Functional API
--	----------------

For text generation, we used a simple **Sequential** model because it has one input and one straight flow:
tokens → Embedding → LSTM → Dense

Translation is different because it needs two connected parts:

English sentence → Encoder → hidden states
French input tokens + hidden states → Decoder → French output tokens

Key Idea

We use the Functional API because the decoder needs both its own input sequence and the hidden states from the encoder.

Seq2Seq Coding: Translation Example—Continued

```
# 10. Decoder LSTM
decoder_inputs = Input(shape=(max_decoder_len - 1,))

decoder_embedding_layer = Embedding(input_dim=french_vocab_size, output_dim=embedding_dim, mask_zero=True)

decoder_embedding = decoder_embedding_layer(decoder_inputs)

decoder_lstm = LSTM(lstm_units, return_sequences=True, return_state=True)

decoder_outputs, _, _ = decoder_lstm(decoder_embedding, initial_state=encoder_states)

decoder_dense = Dense(french_vocab_size, activation="softmax")

decoder_outputs = decoder_dense(decoder_outputs)

# 11. Training model
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)

model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])

model.summary()
```

Functional API for Seq2Seq Translation

The Functional API is used because Seq2Seq translation is not one straight pipeline.

It has two connected inputs:

- **English input** goes to the encoder.
- **French input** goes to the decoder.

English input → Encoder LSTM → final states



French input → Decoder LSTM using encoder states → Dense → French token probabilities

Key Idea

The encoder's final hidden and cell states are passed into the decoder, so the decoder can generate French tokens based on the English sentence.

Token-Level Accuracy in Translation

In Seq2Seq translation, accuracy is measured across token positions.

The model compares the predicted token with the correct target token at each decoder time step, across all samples.

Example:

Target token	Predicted token	Result
je	je	correct
suis	suis	correct
heureux	triste	incorrect

For this example:

$$\text{accuracy} = \frac{2}{3} = 66.7\%$$

Important

The reported accuracy is **token-level accuracy**. It is calculated over decoder token positions inside each batch, then averaged across batches.

Seq2Seq Coding: Translation Example—Continued

```
# 12. Train model
history = model.fit([encoder_input_data, decoder_input_data], decoder_target_data, epochs=300, batch_size=8, verbose=1)

# 13. Build inference encoder model
encoder_model = Model(encoder_inputs, encoder_states)

# 14. Build inference decoder model

# Create two empty input placeholders with the same shape as the LSTM states
decoder_state_input_h = Input(shape=(lstm_units,))
decoder_state_input_c = Input(shape=(lstm_units,))
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]

# the decoder will receive one French token ID at a time
decoder_single_input = Input(shape=(1,))

# The decoder cannot understand raw token IDs directly.
# So this line turns a token ID into an embedding vector.
decoder_single_embedding = decoder_embedding_layer(decoder_single_input)
# decoder_embedding_layer is reused from training.

# This runs the decoder LSTM for one step.
decoder_single_outputs, state_h2, state_c2 = decoder_lstm(decoder_single_embedding, initial_state=decoder_states_inputs)

# These states will be returned by the decoder model.
# Then they will be reused in the next prediction step.
decoder_states_outputs = [state_h2, state_c2]

#The LSTM output is not yet a word probability.
# The dense layer converts it into probabilities over every French word in the vocabulary.
decoder_single_outputs = decoder_dense(decoder_single_outputs)
```

Seq2Seq Coding: Translation Example—Continued

```
# This creates the final inference decoder model.
# It takes: 1.current token 2.previous hidden state h 3.previous cell state c
# It returns: 1.predicted next-token probabilities 2.new hidden state h 3.new cell state c
# decoder_model(current_word, old_h, old_c) -> next_word_probabilities, new_h, new_c
decoder_model = Model([decoder_single_input] + decoder_states_inputs, [decoder_single_outputs] + decoder_states_outputs)
# [decoder_single_input] + decoder_states_inputs means [decoder_single_input] + [decoder_state_input_h, decoder_state_input_c]
# decoder_model = Model(inputs=[decoder_single_input, decoder_state_input_h, decoder_state_input_c],
#     outputs=[decoder_single_outputs, state_h2, state_c2])

# 15. Reverse dictionary for French token IDs
# convert predicted token IDs back into French words
index_to_french = {i: word for word, i in french_tokenizer.word_index.items()}

# get the numeric IDs for the special start and end tokens.
start_token_id = french_tokenizer.word_index["startseq"]
end_token_id = french_tokenizer.word_index["endseq"]
```

Why Do We Create Inference Models?

We are not training a new model.

The inference encoder and decoder reuse the same trained layers, but prediction works differently from training.

During training:

English sentence + full French input $\rightarrow$ full French target

During prediction:

English sentence $\rightarrow$ Encoder states

current French token + previous states $\rightarrow$ next token + updated states

Key Idea

During prediction, we do not already have the full French sentence, so the decoder must generate it one token at a time.

Training Model vs Inference Models

The **training model** learns the weights.

English input + French input $\rightarrow$ French target

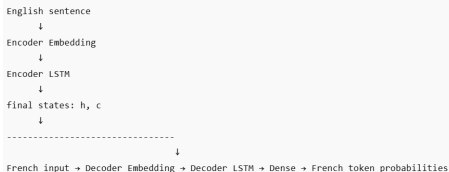
After training, the weights are stored inside the trained layers:

Encoder LSTM, Decoder LSTM, Embedding layers, Dense layer

Then we create **inference models** for prediction:

English sentence $\rightarrow$ Encoder $\rightarrow (h, c)$

current French token + $(h, c) \rightarrow$ Decoder $\rightarrow$ next token + updated (h, c)



Seq2Seq Coding: Translation Example—Continued

```
# 16. Translate one sentence
def translate_sentence(sentence): # This function takes one English sentence and returns a French translation.
    sentence = clean_text(sentence)

    input_seq = english_tokenizer.texts_to_sequences([sentence]) # convert the English sentence into token IDs.
    input_seq = pad_sequences(input_seq, maxlen=max_encoder_len, padding="post")

    # sends the English sentence through the encoder
    states_value = encoder_model.predict(input_seq, verbose=0)

    # The decoder starts with only one input: startseq
    target_seq = np.array([[start_token_id]])

    translated_words = [] # This list stores the predicted French words.

    for i in range(max_decoder_len): # This loop generates one French word at a time.
        # It stops when either: endseq is predicted or max_decoder_len is reached

        # Given the current token and current memory, what word comes next?
        output_tokens, h, c = decoder_model.predict([target_seq] + states_value, verbose=0)

        # The decoder outputs probabilities for all French tokens.
        predicted_token_id = np.argmax(output_tokens[0, -1, :])

        if predicted_token_id == end_token_id:
            break

        # This converts the predicted token ID into an actual French word.
        predicted_word = index_to_french.get(predicted_token_id, "")
        # Find the French word for predicted_token_id.
        # If the ID does not exist, return an empty string instead.

        # This avoids adding unwanted tokens to the final translation.
        if predicted_word not in ["", "startseq", "endseq", "<OOV>"]:
            translated_words.append(predicted_word)

        # The predicted word becomes the next input.
        target_seq = np.array([[predicted_token_id]])

        # The decoder memory gets updated. This allows the decoder to remember what it has already generated.
        states_value = [h, c]

    return " ".join(translated_words)
```

Seq2Seq Coding: Translation Example—Result

```
# 17. Try translation
test_sentences = [
    "i am happy",
    "i love you",
    "thank you",
    "where are you",
    "i want water",
    "this movie is good",
    "i love you"
]

for sentence in test_sentences:
    translation = translate_sentence(sentence)
    print("English:", sentence)
    print("French:", translation)
    print()
```

What is the main problem with classic Seq2Seq RNN models?

Problem

- In the basic seq2seq model, the encoder squeezes the entire input into **one fixed-size vector**.
- This is problematic:
 - **Encoder side:** it's nearly impossible to capture all details of a long sentence in a single vector.
 - **Decoder side:** different parts of the input may matter at different time steps.
- Result: important information can be lost → poor translations, especially for long sentences.

We Are Losing Long-Term Dependencies...

In RNN seq2seq, the entire input sentence is compressed into **one fixed-size vector**.

How can we overcome this bottleneck, especially for long or complex sentences?



We Need More Attention!

The encoder's single vector is too forgetful...

**What if the decoder could look back at the input
whenever it wants?**

That's why we use **Attention!**

We will cover the attention mechanism and how Transformers benefit from it in the next chapter.

From RNNs to Transformers

Limitations of RNNs

- They process text sequentially, token by token.
- This makes them difficult to parallelize.
- They can struggle with very long-range dependencies.
- Their memory can fade as sequence length grows.

Key Idea of Transformers

- Transformers replace recurrence with an attention mechanism.
- They model word relationships simultaneously, not step by step.
- They enable efficient parallel computation.
- They capture long-term context more effectively.

Main Point

Transformers revolutionized NLP by using attention instead of recurrence.

Congratulations!

Now you know how Seq2Seq models work using RNN-based models!

